

Customer Churn Analysis and Prediction

Task 4

Churn Prediction Model, Evaluation & Interpretation

Internship Report

Dataset: Telco Customer Churn Dataset

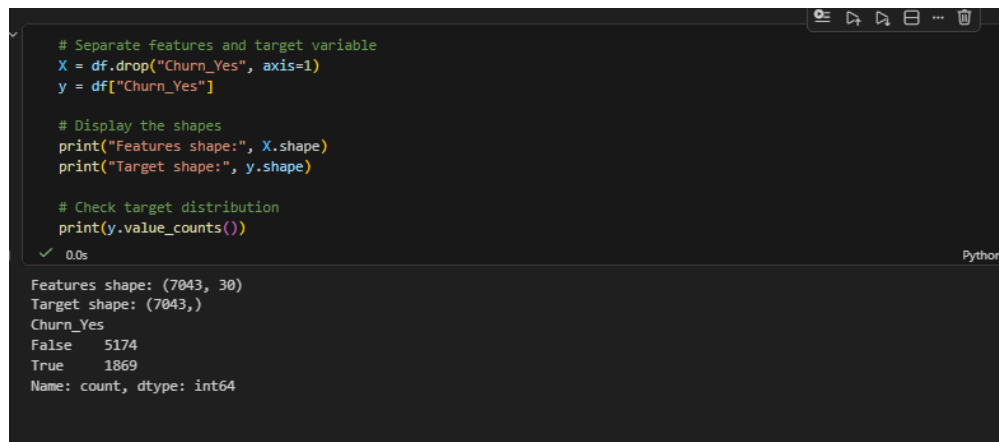
Table of Contents

Table of Contents	2
1. Introduction	3
2. Target Variable Analysis	3
3. Train-Test Split	3
4. Model 1 — Logistic Regression	4
4.1 Example Prediction	4
4.2 Evaluation	5
5. Model 2 — Decision Tree	5
6. Model 3 — Random Forest	6
7. Model Comparison	7
8. Feature Selection	7
9. Hyperparameter Tuning	9
10. Model Evaluation and Interpretation	10
10.1 Confusion Matrix	10
10.2 ROC Curve and AUC	11
10.3 Interpreting the Model's Coefficients	12
Features that increase churn risk	13
Features that reduce churn risk	13
11. Final Conclusion and Business Recommendations	14

1. Introduction

This report covers Task 4 of the Customer Churn Analysis and Prediction project: building, tuning, evaluating and interpreting a churn prediction model. Using the cleaned and encoded dataset from Task 1 and the patterns uncovered in Tasks 2 and 3, the goal here was to choose suitable machine learning algorithms, train and compare several models, select and tune the best one, and then evaluate and interpret its predictions in business terms — including feature importance, the confusion matrix, the ROC curve and AUC, and the model's coefficients.

2. Target Variable Analysis



```
# Separate features and target variable
X = df.drop("Churn_Yes", axis=1)
y = df["Churn_Yes"]

# Display the shapes
print("Features shape:", X.shape)
print("Target shape:", y.shape)

# Check target distribution
print(y.value_counts())
```

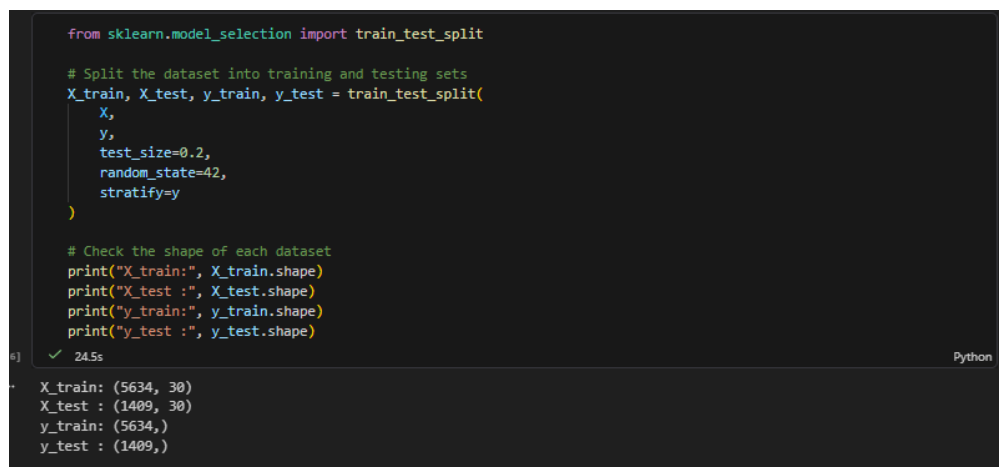
✓ 0.0s Python

```
Features shape: (7043, 30)
Target shape: (7043,)
Churn_Yes
False    5174
True     1869
Name: count, dtype: int64
```

Figure 1. Distribution of the target variable, Churn_Yes.

The prepared dataset contains 7,043 customer records with 30 input features used for training, after one-hot encoding. The target variable, Churn_Yes, is binary: 5,174 customers (73.5%) did not churn, while 1,869 customers (26.5%) churned. This confirms that the dataset is moderately imbalanced, which is why accuracy alone is not a sufficient metric here: a model could reach high accuracy simply by predicting "no churn" for everyone, while completely failing to catch the customers who are actually at risk of leaving. Precision, recall, and F1-score are therefore tracked alongside accuracy throughout this task.

3. Train-Test Split



```
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Check the shape of each dataset
print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

✓ 24.5s Python

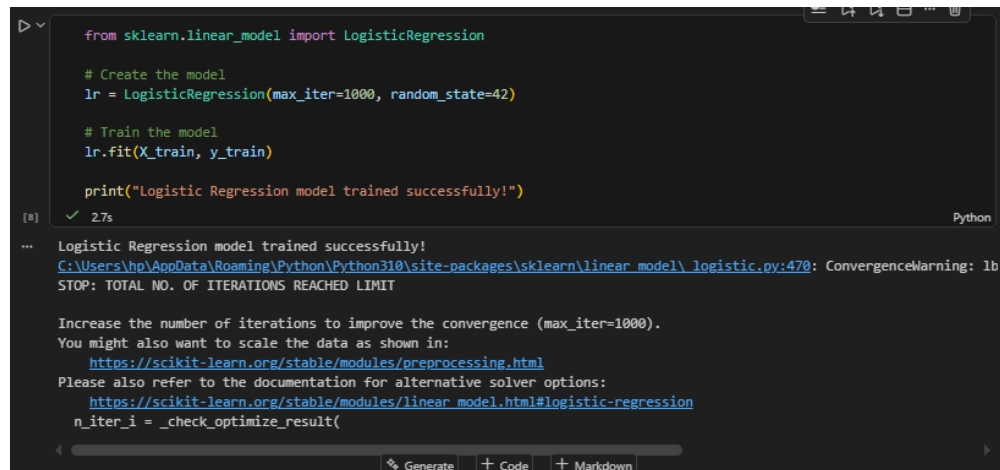
```
X_train: (5634, 30)
X_test : (1409, 30)
y_train: (5634,)
y_test : (1409,)
```

Figure 2. Train-test split code and resulting shapes.

The dataset was split into training and testing sets using an 80:20 ratio (5,634 training records and 1,409 testing records), with `random_state=42` for reproducibility. The `stratify` parameter was applied on the target variable so that both subsets preserve the original 73.5% / 26.5% class balance — an important precaution given the imbalance identified above.

4. Model 1 — Logistic Regression

Logistic Regression was chosen as the baseline model. Although its name contains "regression", it is a classification algorithm well suited to binary problems such as churn vs. no-churn. It is fast, easy to interpret, and provides a solid benchmark against which more complex models can be compared.



```
from sklearn.linear_model import LogisticRegression

# Create the model
lr = LogisticRegression(max_iter=1000, random_state=42)

# Train the model
lr.fit(X_train, y_train)

print("Logistic Regression model trained successfully!")
```

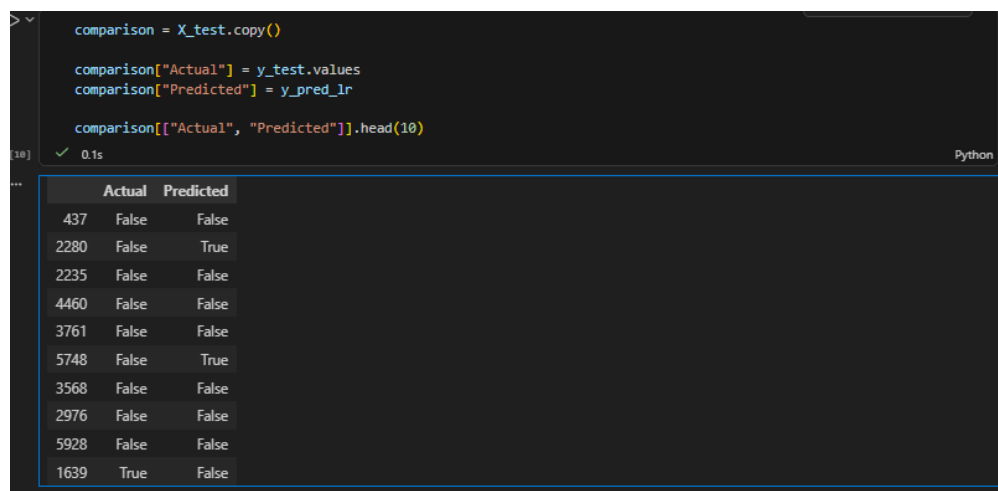
Logistic Regression model trained successfully!
 C:\Users\hp\AppData\Roaming\Python\Python310\site-packages\sklearn\linear_model\logistic.py:470: ConvergenceWarning: lb
 STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
 You might also want to scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
 n_iter_i = _check_optimize_result(

Figure 3. Training the Logistic Regression model.

The model was trained on the training set, with the maximum number of iterations increased to 1,000 to ensure proper convergence.

4.1 Example Prediction



```
comparison = X_test.copy()

comparison["Actual"] = y_test.values
comparison["Predicted"] = y_pred_lr

comparison[["Actual", "Predicted"]].head(10)
```

	Actual	Predicted
437	False	False
2280	False	True
2235	False	False
4460	False	False
3761	False	False
5748	False	True
3568	False	False
2976	False	False
5928	False	False
1639	True	False

Figure 4. Example of an individual prediction.

Looking at individual predictions is useful for understanding the model's real-world impact, not just its aggregate metrics. In one example, a customer who actually churned was predicted by the model to stay. For

a company, this type of mistake (a false negative) is serious, because it means the customer could be lost without any retention action ever being triggered.

4.2 Evaluation

```
from sklearn.metrics import [
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
]

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred_lr)
precision = precision_score(y_test, y_pred_lr)
recall = recall_score(y_test, y_pred_lr)
f1 = f1_score(y_test, y_pred_lr)

# Print results
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-score  : {f1:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr))
```

0.1s Python

```
Accuracy : 0.8048
Precision: 0.6542
Recall   : 0.5615
F1-score : 0.6043

Classification Report:
              precision    recall  f1-score   support

   False      0.85      0.89      0.87      1035
    True      0.65      0.56      0.60       374

 accuracy          0.80      0.80      0.80      1409
  macro avg          0.75      0.73      0.74      1409
 weighted avg          0.80      0.80      0.80      1409
```

Figure 5. Logistic Regression evaluation metrics and classification report.

Logistic Regression Results		
Metric	Value	Interpretation
Accuracy	80.48%	Good overall performance.
Precision	65.42%	When the model predicts churn, it is correct about 65% of the time.
Recall	56.15%	The model identifies about 56% of customers who actually churned.
F1-score	60.43%	Indicates a moderate balance between precision and recall.

Figure 6. Logistic Regression classification report (detail).

On the testing set, Logistic Regression achieved an accuracy of 80.48%, a precision of 65.42% (about two-thirds of customers predicted to churn actually did), a recall of 56.15% (just over half of the customers who actually churned were correctly identified), and an F1-score of 60.43%. This gives a solid baseline, although the moderate recall shows that a meaningful number of churners are still missed, leaving room for improvement.

5. Model 2 — Decision Tree

The second model tested was a Decision Tree classifier. Unlike Logistic Regression, which learns a mathematical relationship between the features and the target, a Decision Tree learns a series of if-then rules that split the data into progressively smaller, more homogeneous groups.

```
from sklearn.tree import DecisionTreeClassifier

# Create the model
dt = DecisionTreeClassifier(random_state=42)

# Train the model
dt.fit(X_train, y_train)

print("Decision Tree model trained successfully!")
```

[12] ✓ 1.3s Python

... Decision Tree model trained successfully!

Figure 7. Training the Decision Tree model.

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

accuracy = accuracy_score(y_test, y_pred_dt)
precision = precision_score(y_test, y_pred_dt)
recall = recall_score(y_test, y_pred_dt)
f1 = f1_score(y_test, y_pred_dt)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-score : {f1:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred_dt))
```

[8] ✓ 0.0s Python

... Accuracy : 0.7260
Precision: 0.4838
Recall : 0.4786
F1-score : 0.4812

Classification Report:

	precision	recall	f1-score	support
False	0.81	0.82	0.81	1035
True	0.48	0.48	0.48	374
accuracy			0.73	1409
macro avg	0.65	0.65	0.65	1409
weighted avg	0.73	0.73	0.73	1409

Figure 8. Decision Tree evaluation metrics.

Trained with default parameters (no tuning) on the same training data, the Decision Tree achieved an accuracy of 72.60%, a precision of 48.38%, a recall of 47.86%, and an F1-score of 48.12% — clearly lower than Logistic Regression across every metric. This is most likely due to overfitting, since the tree was not constrained or tuned in any way.

6. Model 3 — Random Forest

The third model was a Random Forest classifier, an ensemble method that combines many Decision Trees and aggregates their predictions through majority voting. By averaging over many trees, Random Forest generally generalizes better than a single Decision Tree and reduces overfitting.

```

from sklearn.ensemble import RandomForestClassifier

# Create the model
rf = RandomForestClassifier(
    random_state=42
)

# Train the model
rf.fit(X_train, y_train)

print("Random Forest model trained successfully!")

```

[9] ✓ 1.3s Python

... Random Forest model trained successfully!

Figure 9. Training the Random Forest model.

On the testing set, Random Forest reached an accuracy of 78.50%, a precision of 61.79%, a recall of 49.73%, and an F1-score of 55.11% — better than the Decision Tree, but still behind Logistic Regression.

7. Model Comparison

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

accuracy = accuracy_score(y_test, y_pred_rf)
precision = precision_score(y_test, y_pred_rf)
recall = recall_score(y_test, y_pred_rf)
f1 = f1_score(y_test, y_pred_rf)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-score  : {f1:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

```

[11] ✓ 0.0s Python

```

Accuracy : 0.7850
Precision: 0.6179
Recall   : 0.4973
F1-score  : 0.5511

Classification Report:
              precision    recall  f1-score   support

   False       0.83       0.89       0.86       1035
    True       0.62       0.50       0.55        374

   accuracy          0.78       1409
  macro avg       0.72       0.69       0.70       1409
 weighted avg       0.77       0.78       0.78       1409

```

Figure 10. Model comparison across the three algorithms.

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	80.48%	65.42%	56.15%	60.43%
Random Forest	78.50%	61.79%	49.73%	55.11%
Decision Tree	72.60%	48.38%	47.86%	48.12%

Across all three models, Logistic Regression achieved the best overall performance, Random Forest ranked second, and the baseline Decision Tree produced the weakest results on every metric. Based on this comparison, Logistic Regression was selected as the best-performing baseline model and carried forward for feature selection, hyperparameter tuning, and final interpretation.

8. Feature Selection

Before optimizing the chosen model further, I wanted to understand which customer attributes matter most for predicting churn — both to improve interpretability and to check whether some features could be removed without hurting performance. Several feature-selection techniques were considered: correlation analysis (not ideal after one-hot encoding), Chi-Square tests (suitable for categorical features), Recursive Feature Elimination (more advanced), and Random Forest feature importance. I chose Random Forest feature importance as the most appropriate method here, since it directly ranks how much each feature contributed to the model's predictions and is a common, industry-standard approach — even when Random Forest itself is not the final chosen model.

```
import pandas as pd

# Create a DataFrame of feature importances
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf.feature_importances_
})

# Sort features by importance
feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

# Display the top 10 features
print(feature_importance.head(10))
```

Figure 11. Computing feature importance using the Random Forest classifier.

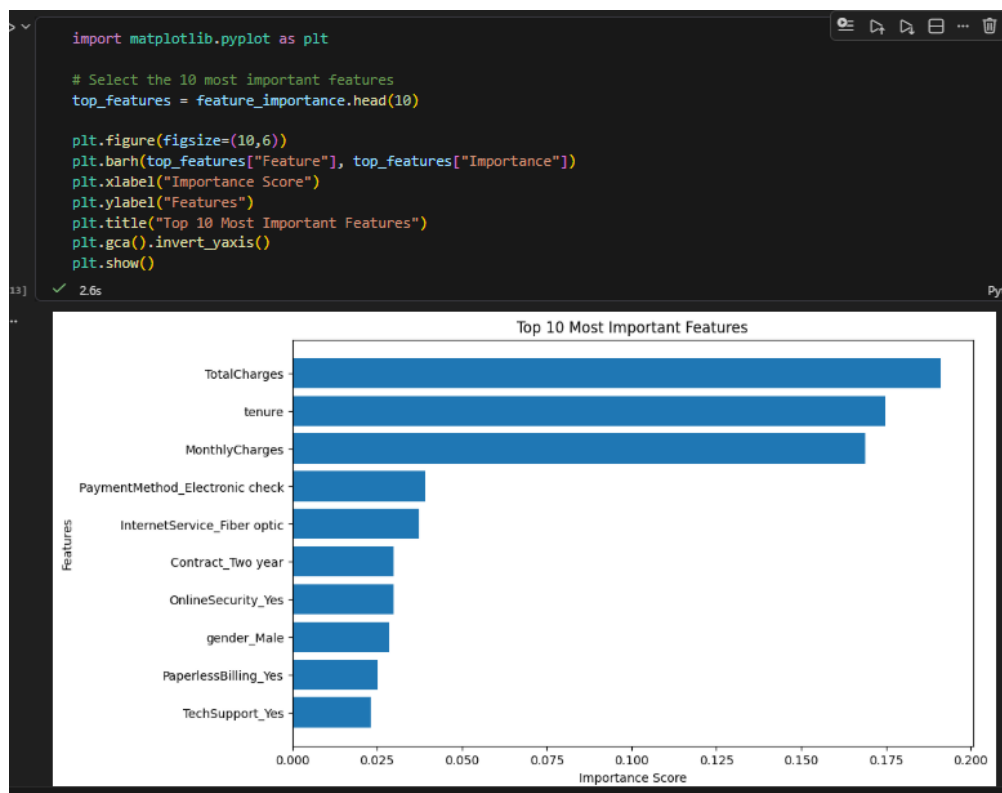


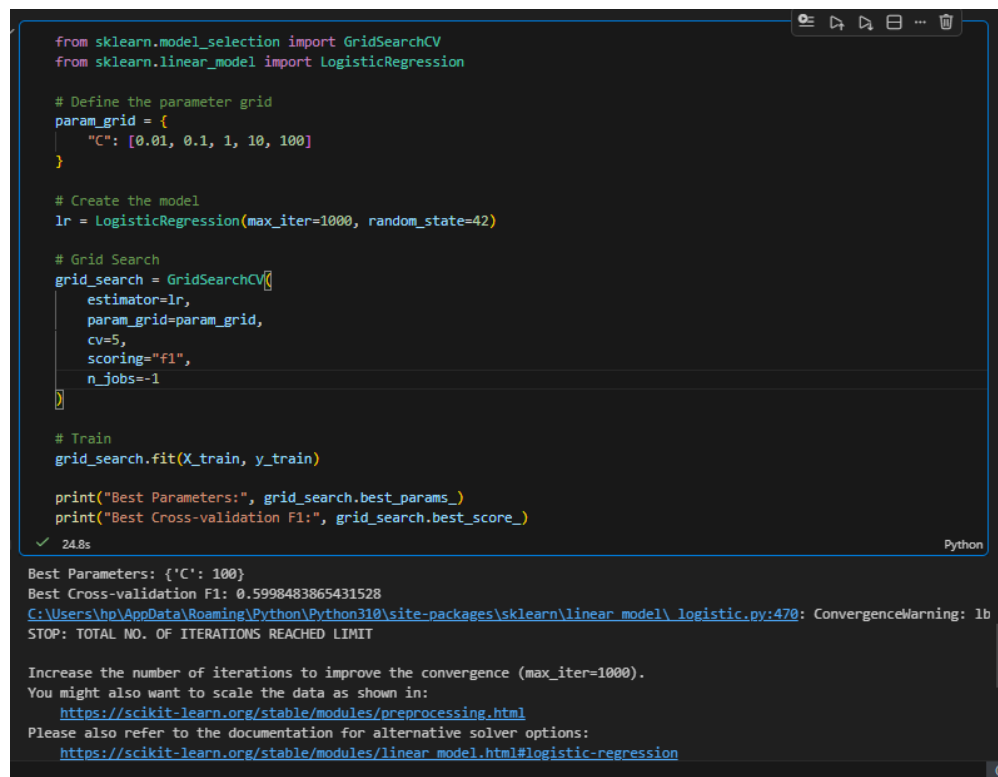
Figure 12. Top features ranked by importance.

The table below summarizes the ten most important features identified.

Rank	Feature	Interpretation
1	TotalCharges	The most influential feature in predicting churn.
2	tenure	Customers with longer tenure are generally less likely to churn.
3	MonthlyCharges	Higher monthly charges can increase the likelihood of churn.
4	PaymentMethod_Electronic check	Electronic check users tend to have higher churn rates.
5	InternetService_Fiber optic	Fiber optic customers show different churn behavior than other internet types.
6	Contract_Two year	Long-term contracts are associated with lower churn.
7	OnlineSecurity_Yes	Customers with online security are generally more loyal.
8	gender_Male	A relatively small contribution to the model.
9	PaperlessBilling_Yes	Some influence on churn prediction.
10	TechSupport_Yes	Customers with technical support tend to churn less frequently.

These findings are consistent with the exploratory data analysis in Task 2, where tenure, contract duration, and monthly charges all showed a strong relationship with churn — reinforcing confidence in both the EDA and the model.

9. Hyperparameter Tuning



```

from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

# Define the parameter grid
param_grid = {
    "C": [0.01, 0.1, 1, 10, 100]
}

# Create the model
lr = LogisticRegression(max_iter=1000, random_state=42)

# Grid Search
grid_search = GridSearchCV(
    estimator=lr,
    param_grid=param_grid,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

# Train
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best Cross-validation F1:", grid_search.best_score_)

```

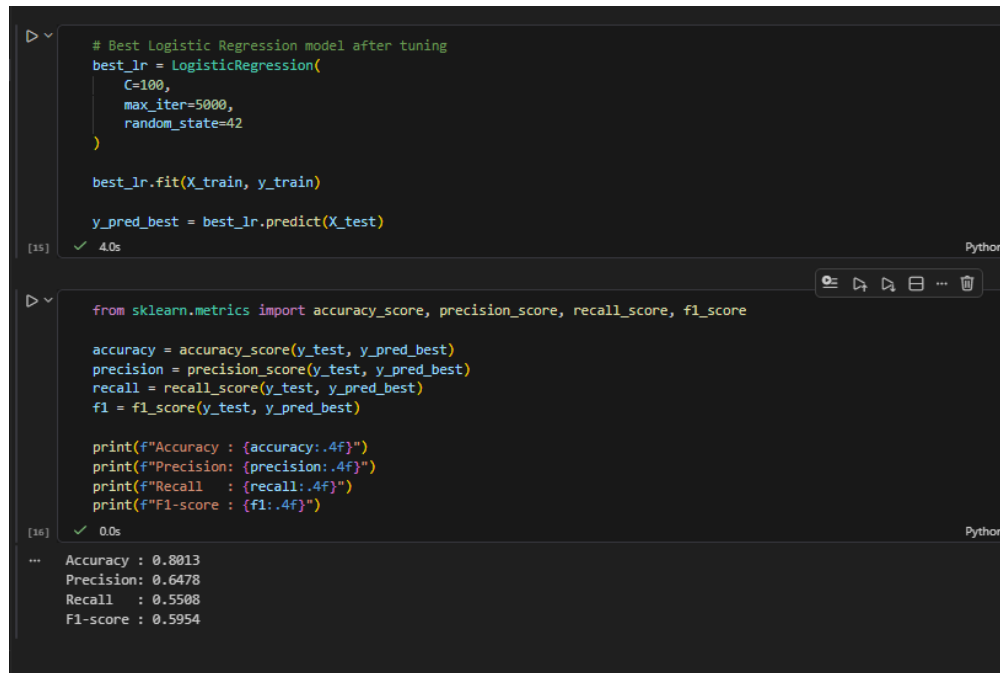
✓ 24.8s Python

Best Parameters: {'C': 100}
Best Cross-validation F1: 0.5998483865431528
C:\Users\hp\AppData\Roaming\Python\Python310\site-packages\sklearn\linear_model\logistic.py:470: ConvergenceWarning: 1b
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

Figure 13. GridSearchCV hyperparameter tuning for Logistic Regression.

Hyperparameter tuning was performed using GridSearchCV to identify the best regularization parameter (C) for the Logistic Regression model. Several values of C were evaluated with 5-fold cross-validation, using F1-score as the optimization metric — a more appropriate choice than accuracy given the class imbalance. The search identified C = 100 as the optimal value.



```

# Best Logistic Regression model after tuning
best_lr = LogisticRegression(
    C=100,
    max_iter=5000,
    random_state=42
)

best_lr.fit(X_train, y_train)

y_pred_best = best_lr.predict(X_test)

[15] ✓ 4.0s

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy = accuracy_score(y_test, y_pred_best)
precision = precision_score(y_test, y_pred_best)
recall = recall_score(y_test, y_pred_best)
f1 = f1_score(y_test, y_pred_best)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-score : {f1:.4f}")

[16] ✓ 0.0s

... Accuracy : 0.8013
Precision: 0.6478
Recall : 0.5508
F1-score : 0.5954

```

Figure 14. Comparing the tuned model against the original model.

After retraining with C = 100 and evaluating on the testing set, the tuned model's performance was very close to the original, untuned Logistic Regression model. As a data scientist, the right choice is the model with the best actual test performance rather than the one that was tuned for its own sake — and since the original model performed at least as well, it was kept as the final model:

Final model — Logistic Regression

Accuracy: 80.48% | Precision: 65.42% | Recall: 56.15% | F1-score: 60.43%

10. Model Evaluation and Interpretation

10.1 Confusion Matrix

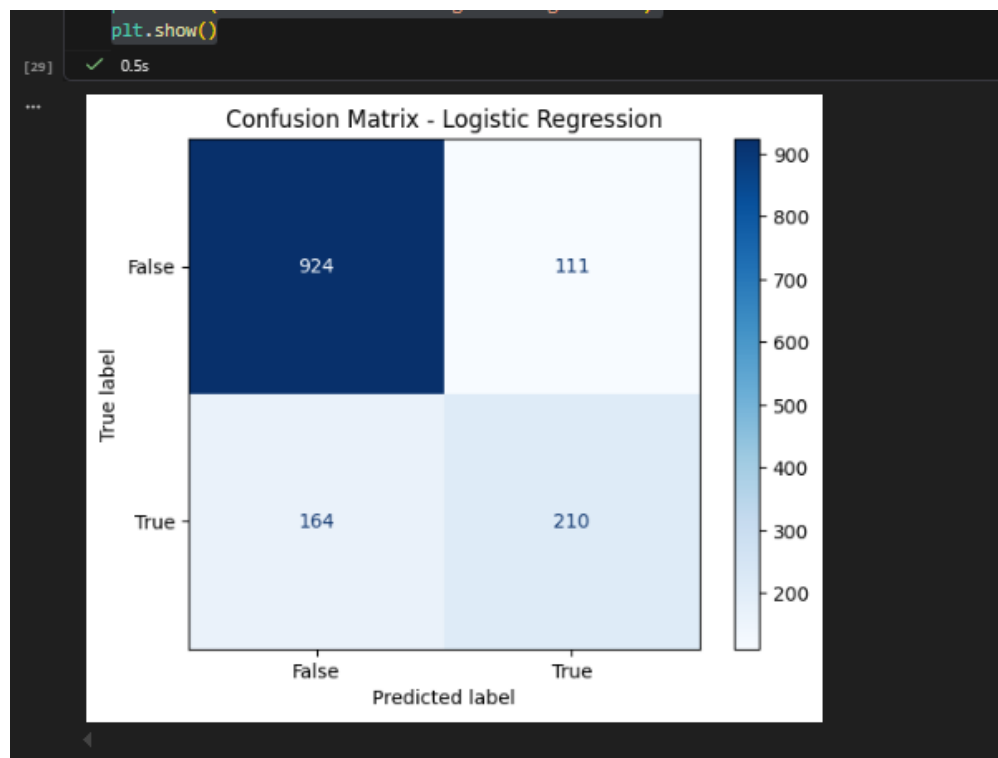


Figure 15. Confusion matrix for the final Logistic Regression model.

The confusion matrix gives a detailed view of the model's predictions on the testing set: 924 non-churning customers were correctly classified (true negatives) and 210 churning customers were correctly identified (true positives). However, 111 customers were incorrectly flagged as likely to churn when they actually stayed (false positives), and 164 customers who eventually churned were missed entirely (false negatives). From a business perspective, the false negatives are the most costly: these are customers who leave without ever receiving a retention offer. Overall, the model is strong at identifying loyal customers, but there is still room to improve detection of customers who are actually at risk.

10.2 ROC Curve and AUC

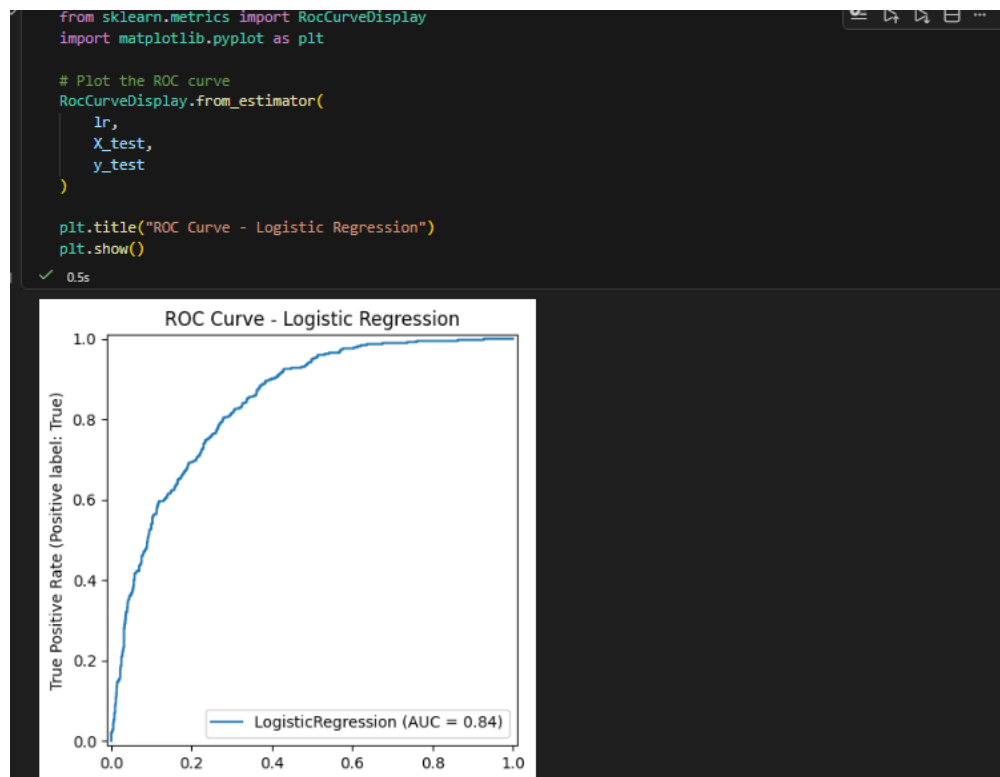


Figure 16. ROC curve for the final Logistic Regression model.

The ROC (Receiver Operating Characteristic) curve evaluates how well the model separates customers who will churn from those who will not, across different decision thresholds, by plotting the true positive rate against the false positive rate. The Area Under the Curve (AUC) obtained was 0.84, which falls in the "good" range (0.80–0.90) and means that, given one random churner and one random non-churner, the model has about an 84% chance of assigning a higher churn probability to the customer who actually churned.

10.3 Interpreting the Model's Coefficients

```

# Sort coefficients
coefficients = coefficients.sort_values(
    by="Coefficient",
    ascending=False
)

# Display the 10 most positive coefficients
print("Top 10 Positive Coefficients:")
print(coefficients.head(10))

print("\nTop 10 Negative Coefficients:")
print(coefficients.tail(10))

```

Top 10 Positive Coefficients:

	Feature	Coefficient
10	InternetService_Fiber optic	0.742896
28	PaymentMethod_Electronic check	0.387787
26	PaperlessBilling_Yes	0.376456
9	MultipleLines_Yes	0.274968
23	StreamingMovies_Yes	0.204574
21	StreamingTV_Yes	0.204150
8	MultipleLines_No phone service	0.164610
0	SeniorCitizen	0.147470
29	PaymentMethod_Mailed check	0.080358
5	Partner_Yes	0.024925

Top 10 Negative Coefficients:

	Feature	Coefficient
18	TechSupport_No internet service	-0.111778
16	DeviceProtection_No internet service	-0.111778
20	StreamingTV_No internet service	-0.111778
15	OnlineBackup_Yes	-0.181573
6	Dependents_Yes	-0.219680
19	TechSupport_Yes	-0.389446
7	PhoneService_Yes	-0.430754
13	OnlineSecurity_Yes	-0.437039
24	Contract_One year	-0.684531
25	Contract_Two year	-1.325214

Figure 17. Logistic Regression coefficients, ranked by impact on churn.

Positive coefficients indicate variables associated with a higher likelihood of churn, while negative coefficients indicate variables associated with retention.

Features that increase churn risk

Feature	Coefficient	Interpretation
InternetService_Fiber optic	0.743	Fiber optic customers are more likely to churn.
PaymentMethod_Electronic check	0.388	Electronic check users have a higher risk of churn.
PaperlessBilling_Yes	0.376	Paperless billing customers are slightly more likely to churn.
MultipleLines_Yes	0.275	Having multiple phone lines slightly increases churn probability.
StreamingMovies_Yes	0.205	A small positive relationship with churn.
StreamingTV_Yes	0.204	A small positive relationship with churn.
SeniorCitizen	0.147	Senior customers show a slightly higher tendency to churn.

Features that reduce churn risk

Feature	Coefficient	Interpretation
Contract_Two year	-1.325	The strongest factor reducing churn.

Feature	Coefficient	Interpretation
Contract_One year	-0.685	Also significantly reduces churn.
OnlineSecurity_Yes	-0.437	Customers with online security are less likely to churn.
PhoneService_Yes	-0.431	Associated with lower churn.
TechSupport_Yes	-0.389	Technical support improves customer retention.
Dependents_Yes	-0.220	Customers with dependents are less likely to leave.

The strongest retention factor by far is a two-year contract, which matches the exploratory findings from Task 2 and the segmentation results from Task 3. Tech support and online security also stand out as value-added services that meaningfully improve retention, while fiber optic internet, electronic-check payment, and paperless billing are the strongest factors pushing customers toward churn.

11. Final Conclusion and Business Recommendations

Logistic Regression was selected as the final churn prediction model, reaching 80.48% accuracy and an AUC of 0.84 — a good level of discrimination between customers who will churn and those who will not. Feature importance and coefficient analysis both converge on the same story that emerged from the EDA and segmentation tasks: contract length, tenure, and monthly charges are the dominant drivers of churn, with fiber optic internet, electronic-check payment, and the absence of tech support or online security further increasing risk.

- Prioritize converting month-to-month customers to one- or two-year contracts, especially fiber optic and electronic-check customers.
- Use the model's predicted churn probability to proactively target at-risk customers — particularly to reduce false negatives, which are the costliest errors for the business.
- Bundle tech support and online security with at-risk segments, since both are strongly associated with retention.
- Focus onboarding and early support efforts on the first few months of the customer relationship, where churn risk is highest.